```
/-
Copyright (c) 2026 Filippo A. E. Nuccio. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Filippo A. E. Nuccio
-/

module

public import LFTCM2026.Preliminaries

/- # Algebra (and Number Theory)
The goal of today's lecture is to discuss some algebraic structures: (Sub)Groups, Rings and Vector
Spaces. If time permits (and it won't) we might discuss a bit of modular arithmetic.
-/

section Groups

/- ## Groups
We've already seen in the last lecture how to *define* groups. Let's see how to play with them.
-/

example {G : Type*} [Group G] (x y z : G) : x * (y * z) * (x * z)⁻¹ * (x * y * x⁻¹)⁻¹ = 1 := by
  group -- this works for **free groups**
  done


example {G : Type*} [CommGroup G] (x y : G) : (x * y)⁻¹ = x⁻¹ * y⁻¹ := by
  -- group -- the group is commutative!
  rw [mul_inv_rev, mul_comm]
  -- rw [mul_inv]
  done

example {A : Type*} [AddCommGroup A] (x y : A) : x + y + 0 = x + y := by
  abel
```

```
35    done
36
37  /- All this is very nice, but are we *duplicating* the whole
    library for both additive and
38  multiplicative groups? -/
39  -- -- whatsnew in
40  -- @[to_additive]
41  lemma mul_square {G : Type*} [Group G] {x y : G} (h : x * y
    = 1) : x * y ^ 2 = y := by
42    rw [pow_two, ← mul_assoc, h]
43    simp
44    done
45
46  example {A : Type*} [AddGroup A] {a b : A} (h : a + b = 0) :
    a + 2 • b = b := by
47    -- exact add_even h -- uncomment @[to_additive]
48    rw [two_nsmul, ← add_assoc, h]
49    simp
50    done
51
52  /- ### Subgroups
53  Given all the stress I put in saying **we must use
    structures**, you might imagine how we
54  begin to define what a subgroup is...
55  -/
56  #print Subgroup
57  /-
58  structure Subgroup.{u_3} (G : Type u_3) [Group G] : Type u_3
59  number of parameters: 2
60  parents:
61    Subgroup.toSubmonoid : Submonoid G
62  fields:
63    Subsemigroup.carrier : Set G
64    Subsemigroup.mul_mem' : ∀ {a b : G}, a ∈ self.carrier → b
    ∈ self.carrier → a * b ∈ self.carrier
65    Submonoid.one_mem' : 1 ∈ self.carrier
66    Subgroup.inv_mem' : ∀ {x : G}, x ∈ self.carrier → x⁻¹ ∈
    self.carrier
67  constructor:
68    Subgroup.mk.{u_3} {G : Type u_3} [Group G] (toSubmonoid :
    Submonoid G)
```

```
69      (inv_mem' : ∀ {x : G}, x ∈ toSubmonoid.carrier → x⁻¹ ∈
    toSubmonoid.carrier) : Subgroup G
70  field notation resolution order:
71    Subgroup, Submonoid, Subsemigroup
72
73  Concretely, there are *four* fields: a `carrier` (which is a
    set in `G`) and three proofs that
74  this `carrier` is stable with respect to the operations. So
    **a subgroup is a quadruple**:
75  -/
76  variable (G : Type*) [Group G] (H : Subgroup G)
77  #check H.carrier
78  #check H.mul_mem'
79  #check H.one_mem'
80  #check H.inv_mem'
81
82  -- And, hopefully, a subgroup is *also* a group:
83  example (H : Subgroup G) : Group H := inferInstance
84
85  variable (H : Subgroup G) in
86  #synth Group H
87
88  -- The following two examples seem to say something stupid,
    but for Lean it's an effort!
89  example (H : Subgroup G) (x : H) (hx : x = 1) : (x : G) = 1
    := by -- are the two `1`'s the same?
90    simp [hx]
91    done
92
93  example (H : Subgroup G) : 1 ∈ H := H.one_mem
94
95  -- Let's define `2ℤ` as a term of `Subgroup ℤ`... (almost
    doable!)
96  example : AddSubgroup ℤ where
97    carrier := {n : ℤ | Even n}
98    add_mem' := by
99      intro a b ha hb
100     -- simp at ha hb --not needed, actually
101     simp only [Even] at ha hb
102     obtain ⟨m, hm⟩ := ha
103     obtain ⟨n, hn⟩ := hb
104     rw [hn, hm]
```

```
    use n + m
    abel
    -- grind
    done
  zero_mem' := ⟨0, by abel⟩
  neg_mem' {x} hx := by
    obtain ⟨r, _⟩ := hx
    exact ⟨-r, by simp_all⟩
    done

/- In the example below, note two things:
1. What happens if we remove `Comm`;
2. What happens to the `G` and `Group G` that are globally defined in this section;
-/
example (G : Type*) [CommGroup G] (H₁ H₂ : Subgroup G) {x y : G} (hx : x ∈ H₁) (hy : y ∈ H₂) :
    x * y ∈ H₁ ⊔ H₂ := by
  rw [Subgroup.mem_sup]
  use x, hx, y, hy
  done


---Let's discuss **dot notation**.
example : (Subgroup.center G).Normal := by
-- #print Subgroup.Normal
  apply Subgroup.Normal.mk
  intro z hz g
  let hz' := hz
  rw [Subgroup.mem_center_iff] at hz --this looses hz
  specialize hz g
  rw [← mul_inv_eq_iff_eq_mul] at hz
  rwa [hz]
  -- exact Subgroup.instNormalCenter
  done

/- ### Homomorphisms
Once again, a homomorphism is a collection of things: a bare function, together with the properties
that it must satisfy to be a group homomorphism. So, the *collection* of all these homomorphisms
```

```
142 is a `structure`, denoted `G →* H`! Note that actually we
    define `monoid` homomorphisms, because a
143 multiplicative map sending `1` to `1` automatically
    preserves the inverse. -/
144
145 structure MonoidHom_Cortona (M N : Type*) [Monoid M] [Monoid
    N] where
146   toFun : M → N
147   map_one : toFun 1 = 1
148   map_mul : ∀ (x y : M), toFun (x * y) = (toFun x) * (toFun
    y)
149
150
151 /- Two examples of things we can say about a morphism: -/
152 -- whatsnew in
153 -- @[to_additive __]
154 example (G₁ : Type) [CommGroup G₁] (f : G →* G₁) : ∀ x y :
    G, x * y = 1 → (f x) * (f y) = 1 := by
155   intro x y h
156   rw [← map_mul, h, map_one]
157   done
158
159 example (A : Type*) [AddGroup A] (f : G →* A) : ∀ x y : G, x
    * y = 1 → (f x) * (f y) = 1 := by
160   sorry
161   done
162
163
164 end Groups
165
166 section Rings
167
168 /- ## Rings
169 The situation is very similar to the one for `Group`s:
    they're a class, and they have a dedicated
170 tactic, `ring`: it closes every goal that holds in a *free
    commutative ring*. There is also `grind`
171 that, on top of `ring` , is capable of performing (some)
    logical reasoning and arithmetic,
172 including inequalities. -/
173
174 #synth Ring ℤ
```

```
175 #synth CommRing ℤ
176 #synth Monoid ℤ
177 #synth AddMonoid ℤ
178 #check CommRing.toCommMonoid
179 #check CommRing.toAddCommGroupWithOne
180
181 example (a b c : ℚ) : a * (b + c * 1) = b * a + a * c := by
182   -- rw [mul_one]
183   -- rw [← add_assoc]
184   ring
185   done
186
187
188 #check mul_one
189 #check add_assoc
190 #print AddMonoid
191 #print Field
192 #print IsDomain
193 #print CommMonoid
194
195
196 example (R : Type*) [CommRing R] (x y : R) : (x + y) ^ 2 = x
    ^ 2 + 2 * x * y + y ^ 2 := by
197   ring
198   -- exact? ==> exact add_pow_two x y
199   done
200
201 lemma sixth_pow (R : Type*) [CommRing R] (x y : R) : (x + y)
    ^ 6 =
202     x ^ 6 + 6 * x ^ 5 * y +  15 * x ^ 4 * y ^ 2 + 20 * x ^ 3
    * y ^ 3 +
203       15 * x ^ 2 * y ^ 4 + 6 * x * y ^ 5 + y ^ 6 := by
204   -- exact?
205   -- grind
206   ring
207   done
208
209
210 example (R : Type*) [CommRing R] (x y : R) (H : x ^ 2 ≠ y ^
    2) : x ≠ y := by
211   -- ring
212   grind
```

```
213   done
214
215 variable {R S : Type*} [CommRing R] [CommRing S]
216
217 example (f : R →+* S) (r : R) : IsUnit r → IsUnit (f r) :=
    by
218   unfold IsUnit
219   -- use r -- the arrow `↑u` is there for a reason...
220   intro h
221   -- obtain ⟨v, hv⟩ := h -- probably not enough: what does
    it mean `v : Rˣ`? Let's hover on it...
222   obtain ⟨⟨v, u, hvu, huv⟩, H⟩ := h
223   -- simp only at H
224   fconstructor
225   · use f r
226     · use (f u)
227     · rw [← map_mul, ← H, hvu, map_one]
228     · rw [← map_mul, ← H, huv, map_one]
229   · simp
230   done
231
232 end Rings
233
234 section VectorSpaces
235
236 /- We all agree that, at the end of the day, a vector space
    `V` over a field `K` is just a
237 `K`-module, but it is much better to call it a *`K`-vector
    space* rather than *`K`-module`...
238
239 Except that we **don't agree**.
240 -/
241
242 #check VectorSpace
243 #check vectorSpace
244 #check Vectorspace
245 #check Module
246
247 variable (K : Type*) [Field K]
248 #print Module --it's a class
249
250 variable (V : Type*) /- [AddCommGroup V]  -/[Module K V]
```

```
--it *requires* that `V` be a(n additive, commutative) group:


example (T : Submodule K V) (x y : V) (c : K) : x ∈ T → y ∈ T →
  -- c * x + y ∈ T := sorry -- **why this does not work?** The `*` should be a `•`
    c • x + y ∈ T := by
  intro hx hy
  -- exact T.add_mem (T.smul_mem c hx) hy
  apply Submodule.add_mem
  apply Submodule.smul_mem
  assumption
  assumption
  done


example (f : K →ₗ[K] K) : ∃ c, f = (fun x ↦ c • x) := by -- what is going on?
-- example (f : K →ₗ[K] K) : ∃ c, f = .mulLeft K c := by
--what happens without the `.` in `mulLeft`? Why?
  use f 1
  ext
  simp
  done

/- Let's try to state that "the collection of linear maps from `V` to `W` that vanish on a
subspace `T ≤ V` form a linear/vector subspace of all linear maps.
-/
variable (W : Type*) [AddCommGroup W] [Module K W] in
-- theorem Annihilator_Submodule (T : Subspace K V) :
--   Submodule K {f : V →ₗ[K] W | ∀ x ∈ T, f x = 0} := sorry
/- **Indeed**: it must be a `def`: note, in passing, that Lean accepts that `(V →ₗ[K] W)`
  is a module. -/
def AnnihilatorSubmodule (T : Subspace K V) : Submodule K (V →ₗ[K] W) where
  carrier := {f : V →ₗ[K] W | ∀ x ∈ T, f x = 0}
  add_mem' {f g} hf hg x hx := by
```

```
285     simp at hf hg ⊢
286     grind
287     done
288   zero_mem' := by simp
289   smul_mem' c {f} hf := by
290     simp_all
291     done
292
293
294 end VectorSpaces
295
296
297 section ZModn
298
299 /- ##ZMod
300 We don't have time in this class to discuss how *quotients* are defined, but you can easily imagine
301 that everything exists (and probably will have something to do with `structure`s and `class`es...)
302 Just to get a feeling, here are three small examples of results in `ℤ/nℤ`. -/
303
304 lemma exists_b_mul_zero (a n : ℕ) (H : ¬ Nat.Coprime a n) : ∃ b : ℕ, (a * b : (ZMod n)) = 0 := by
305   rw [Nat.Coprime, Nat.gcd_eq_one_iff] at H
306   push Not at H
307   -- obtain ⟨c, hc₁, hc₂, hc₃⟩ := H
308   obtain ⟨c, ⟨b₁, hb₁⟩, ⟨b₂, hb₂⟩, hc₁⟩ := H
309   rw [hb₁]
310   use b₂
311   rw_mod_cast [mul_assoc, mul_comm b₁, ← mul_assoc, ← hb₂]
312   rw [ZMod.natCast_eq_zero_iff]
313   use b₁
314   done
315
316 example (a n : ℕ) (H : ¬ Nat.Coprime a n) : ∃ b, (a * b : ℤ ⧸ (Ideal.span {(n : ℤ)})) = 0 := by
317   obtain ⟨b, hb⟩ := exists_b_mul_zero a n H
318   use b
319   -- exact hb
320   let φ := Int.quotientSpanEquivZMod n
321   apply_fun φ
```

```
322   simpa
323   done
324
325 example (a n : ℕ) (H : ¬ Nat.Coprime a n) : ∃ b : ℕ,
326     ((a * b) : ℤ ⧸ (Ideal.span {(n : ℤ)})) = 0 := by
327   suffices h : ∃ b : ℕ, Ideal.Quotient.mk (Ideal.span {(n :
    ℤ)}) ((a * b) : ℤ) = 0 by
328     obtain ⟨b, hb⟩ := h
329     use b
330     exact hb
331   simp_rw [Ideal.Quotient.eq_zero_iff_dvd]
332   rw [Nat.Coprime, Nat.gcd_eq_one_iff] at H
333   push Not at H
334   obtain ⟨c, ⟨b₁, hb₁⟩, ⟨b₂, hb₂⟩, hc₁⟩ := H
335   rw [hb₂, hb₁]
336   exact ⟨b₂, b₁, by grind⟩
337   done
338
339
340 end ZModn
341
342
343 noncomputable section Exercises
344 open Function
345
346
347 /- **¶ Exercise**
348 Why is the following example broken? Fix its statement, then
    prove it. -/
349 example (G : Type*) [Group G] (H₁ H₂ : Subgroup G) :
    Subgroup (H₁ ∩ H₂) := sorry
350 /- **Sol.:** The error comes come the fact that "being a
    subgroup" is not a proposition. It is the
351 definition of some term! A solution would be -/
352 example (G : Type*) [Group G] (H₁ H₂ : Subgroup G) :
    Subgroup (G) where
353   carrier := H₁ ∩ H₂
354
355
356 -- **¶ Exercise**
357 open Function in
```

```
example (A : Type*) [AddGroup A] (f : A →+ ℤ) (hf : 1 ∈ f.range) : Surjective f := by
  rcases hf with ⟨b, hb⟩
  intro m
  use m • b
  simp [map_zsmul, hb]
  done


/- **¶ Exercise**
Prove the claim made in class that a monoid homomorphism between groups respects the inverse. -/
example (G H : Type*) [Group G] [Group H] (f : MonoidHom G H) (x : G) : f x⁻¹ = (f x)⁻¹ := by
  rw [← mul_eq_one_iff_eq_inv, ← f.map_mul, inv_mul_cancel, f.map_one]
  done

-- **¶ Exercise**
/- The kernel of a ring homomorphism is an ideal: what is an ideal is part of the exercise... -/
def kernel (R S : Type*) [CommRing R] [CommRing S] (f : R →+* S) : Ideal R where
  carrier := {r : R | f r = 0}
  add_mem' := by
    intro a b ha hb
    simp only [Set.mem_ofPred_eq, map_add]
    rw [hb, ha, zero_add]
    done
  zero_mem' := by
    apply map_zero
    done
  smul_mem' := by
    intro c x hx
    simp only [smul_eq_mul, Set.mem_ofPred_eq, map_mul]
    rw [hx, mul_zero]
    done

/- **¶ Exercise**
State and that a group is commutative if and only if the map `x ↦ x⁻¹` is a group homomorphism:
```

```
even if you find a one-line proof, try to produce the whole term. To get `⁻¹`, type `\-1`. It can
be easier to split this `if and only if` statement in two declarations: are they both lemmas, both
definitions, a lemma and a definition?. -/
def inv_hom_of_comm (G : Type*) [CommGroup G] : G →* G where
  toFun := (·)⁻¹
  map_one' := by simp
  map_mul' x y := by
    simp [← mul_inv_rev, mul_comm]
    done

def comm_of_inv_hom (G : Type*) [Group G] (f : G →* G) (hf : ∀ x, f x = x⁻¹) :
    CommGroup G where
  mul_comm g h := by
    have h1 := hf (g * h)
    rwa [mul_inv_rev, map_mul, hf, hf, inv_mul_eq_iff_eq_mul, ← mul_assoc, eq_mul_inv_iff_mul_eq,
      eq_mul_inv_iff_mul_eq, mul_assoc,
inv_mul_eq_iff_eq_mul] at h1
    done

/- **¶ Exercise**
Prove that the homomorphisms between commutative monoids have a structure of commutative monoid. -/
example (M N : Type*) [CommMonoid M] [CommMonoid N] :
CommMonoid (M →* N) where
  npow_zero := by simp
  npow_succ := by
    intros
    rw [Monoid.npow_succ]
    done
  mul := by
    intro f g
    fconstructor
    · use fun x ↦ f x * g x
      simp
    · intro x y
      simp [map_mul, mul_assoc _ (f y) _, mul_comm (f y) _,
← mul_assoc]
    done
```

```
426   mul_assoc := by
427     intro f g h
428     ext x
429     simp only [MonoidHom.mul_apply]
430     exact mul_assoc ..
431     done
432   one := by
433     fconstructor
434     · use fun _ ↦ 1
435     · intro x y
436       simp
437     done
438   one_mul := by simp
439   mul_one := by simp
440   mul_comm := by
441     intro f g
442     ext x
443     simp only [MonoidHom.mul_apply]
444     exact mul_comm ..
445     done
446
447
448 /- **¶ Exercise**
449 Prove that an injective and surjective group homomorphism is
    an isomorphism:
450 but what's an isomorphism? -/
451 def IsoOfBijective (G H : Type*) [Group G] [Group H] (f : G
    →* H)
452     (h_surj : Surjective f) (h_inj : Injective f) : G ≃* H
    := by
453   set g : G ≃ H := by
454     apply Equiv.ofBijective f
455     simp [Bijective, h_inj, h_surj] with hg
456   use g
457   intro x y
458   simp only [hg, Equiv.toFun_as_coe]
459   grind
460   done
461
462 /- **¶ Exercise**
463 State and prove that the image of an ideal through a
    surjective ring homomorphism is an ideal. -/
```

```
def image_ideal {R S : Type*} [CommRing R] [CommRing S] (f : R →+* S) (I : Ideal R)
    (hf : Surjective f) : Ideal S where
  carrier := f.1 '' I
  add_mem' := by
    intro x y hx hy
    obtain ⟨a, ha_mem, hax⟩ := hx
    obtain ⟨b, hb_mem, hby⟩ := hy
    simp only [RingHom.toMonoidHom_eq_coe, MonoidHom.coe_coe, Set.mem_image, SetLike.mem_coe]
    use a + b
    constructor
    · apply I.add_mem
      · exact ha_mem
      · exact hb_mem
    · rw [map_add, ← hax, ← hby]
      simp
    done
  zero_mem' := by
    simp only [RingHom.toMonoidHom_eq_coe, MonoidHom.coe_coe, Set.mem_image, SetLike.mem_coe]
    use 0
    constructor
    · apply I.zero_mem
    apply map_zero
    done
  smul_mem' := by
    intro s x hx
    obtain ⟨a, ha_mem, hax⟩ := hx
    obtain ⟨r, hr⟩ := hf s
    simp only [RingHom.toMonoidHom_eq_coe, MonoidHom.coe_coe, smul_eq_mul, Set.mem_image,
      SetLike.mem_coe]
    use r * a
    constructor
    · apply I.mul_mem_left
      exact ha_mem
    · rw [map_mul, hr, ← hax]
      rfl
    done

/- **¶ Exercise**
```

```
This is the standard fact that an ideal containing a unit is the whole ring. -/
example {R S : Type*} [CommRing R] [CommRing S] (I : Ideal R) (r : R) :
    r ∈ I → IsUnit r → I = ⊤ := by
  intro h_mem h
  obtain ⟨⟨u, v, huv, hvu⟩, H⟩ := h
  simp only at H
  ext x
  constructor
  · intro hx
    simp only [Submodule.mem_top]
  · intro hx
    set y := r * x with hy
    have : y ∈ I := by
      rw [hy]
      rw [mul_comm]
      apply I.mul_mem_left
      exact h_mem
    apply_fun (fun t ↦ v • t) at hy
    rw [← H] at hy
    rw [smul_eq_mul, smul_eq_mul, ← mul_assoc _ u, hvu, one_mul] at hy
    rw [← hy]
    -- rw [← smul_eq_mul]
    -- apply I.smul_mem
    apply I.mul_mem_left
    exact this
  done


/- **¶ Exercise**
**State** and **show** that a linear map is injective if and only if it has trivial kernel: try
first to do the whole proof by hand, and then to find it in Mathlib. -/
example (K V W : Type*) [Field K] [AddCommGroup V] [AddCommGroup W] [Module K V] [Module K W]
    (f : V →ₗ[K] W) : Injective f ↔ f.ker = ⊥ := by
    -- (f.ker_eq_bot_iff).symm
  refine ⟨fun h ↦ ?_, fun h ↦ ?_⟩
  · ext x
```

```
538      -- simp [Submodule.mem_bot]
539      refine ⟨fun hx ↦ ?_, fun hx ↦ ?_⟩
540      · apply h
541        rw [map_zero]
542        apply hx
543      · rw [hx]
544        apply zero_mem
545    · intro x y H
546      rwa [← sub_eq_zero, ← map_sub, ← f.mem_ker, h,
    Submodule.mem_bot, sub_eq_zero] at H
547
548
549  -- **¶ Exercise**
550  variable (K V W : Type*) [Field K] [AddCommGroup V]
    [AddCommGroup W] [Module K V] [Module K W] in
551  /- Ok, this exists in the library: can we understand it in
    detail? In other words: write it
552  down by hand after understanding what all this is about. -/
553  example : SMul K (V →ₗ[K] W) := by
554    constructor
555    intro c f
556    exact {
557    toFun := fun v ↦ c • f v
558    map_add' v w := by simp
559    map_smul' x v := by rw [map_smul, RingHom.id_apply,
    smul_comm]}
560    done
561
562
563  -- **¶ Exercise**
564  /- For the following exercise, you might need `erw` which is
    a stronger version of `rw`, in
565  case you **really really* think that `rw` should work, but
    it's not working. Similarly, consider
566  the tactic `rw_mod_cast` when you want to rewrite something
    but some `↑` is preventing you. All in
567  all, the exercise is a bit of a fight around the problem
    that `ℕ` and `ℤ` differ. -/
568  example (p q : ℕ) (hp : Nat.Prime p) (hq : Nat.Prime q) (hpq
    : p ≠ q) :
569      IsUnit (p : (ℤ ⧸ Ideal.span {(q : ℤ)})) := by
```

```
  have : Nat.Coprime p q := (Nat.coprime_primes hp hq).mpr hpq
  rw [← Nat.isCoprime_iff_coprime, IsCoprime] at this
  obtain ⟨a, b, H⟩ := this
  apply IsUnit.of_mul_eq_one ↑a
  apply_fun (Ideal.Quotient.mk (Ideal.span {(q : ℤ)})) at H
  simp only [eq_intCast, Int.cast_add, Int.cast_mul, Int.cast_natCast, Int.cast_one] at H
  erw [mul_comm, ← H, left_eq_add, Ideal.Quotient.eq_zero_iff_dvd]
  simp
  done


end Exercises

```